Hydroficient IoT Cyber Defense
Project 7: Defeating Replay Attacks

Step 1: Dashboard Architecture

Project Overview
This project brings everything together: **visualize, validate, present**.
By the end of this project, you'll have:

1. **A real-time security dashboard** — a browser-based interface that shows live sensor data, color-coded alerts, and attack statistics from your defended pipeline
2. **A live attack demo** — you'll run a three-phase attack simulator (eavesdrop, inject, replay) and watch every attack get blocked on the dashboard in real time
3. **A customized dashboard** — you'll modify the branding, colors, and alert rules to make the dashboard your own, including an AI-assisted styling exercise
4. **A capstone presentation** — after you complete the dashboard and attack demo, you'll build a capstone presentation — your final deliverable for the externship program. This is separate from the work Maya assigns you this week. Step 3 walks you through a presentation template, AI-assisted slide writing, and how to structure your findings for a non-technical audience. The presentation is structured using the pyramid principle and the Situation-Complication-Resolution framework that consulting firms use, backed by your own experiment data and dashboard screenshots.

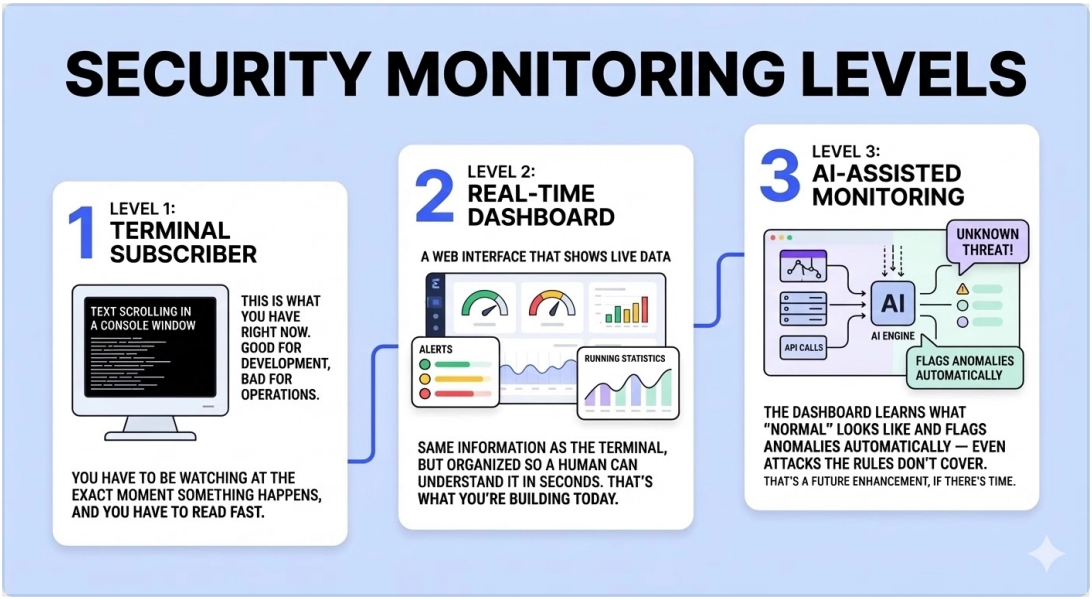
💼 **Career note:** Security Operations Centers (SOCs) — the rooms where analysts monitor dashboards 24/7 — are one of the fastest-growing areas in cybersecurity. Companies like CrowdStrike, IBM, and Palo Alto Networks run SOCs watching millions of events per day. The ability to build monitoring tools, run live attack demos, and present findings to non-technical stakeholders is exactly what hiring managers look for in junior security engineers. After this project, you'll have a working dashboard, a live attack demo recording, and a board-ready presentation in your portfolio.

Why Dashboards Matter
One night at 3 AM, my phone buzzes — "PRESSURE ALERT — INTAKE VALVE 7." I open my laptop, connect to the subscriber. Scrolling text, hundreds of log lines. I can see messages are flowing, but I can't tell at a glance what happened, when it started, or whether it's still happening. By the time I find the relevant line, twenty minutes have passed. That's the problem with terminal output. It works for

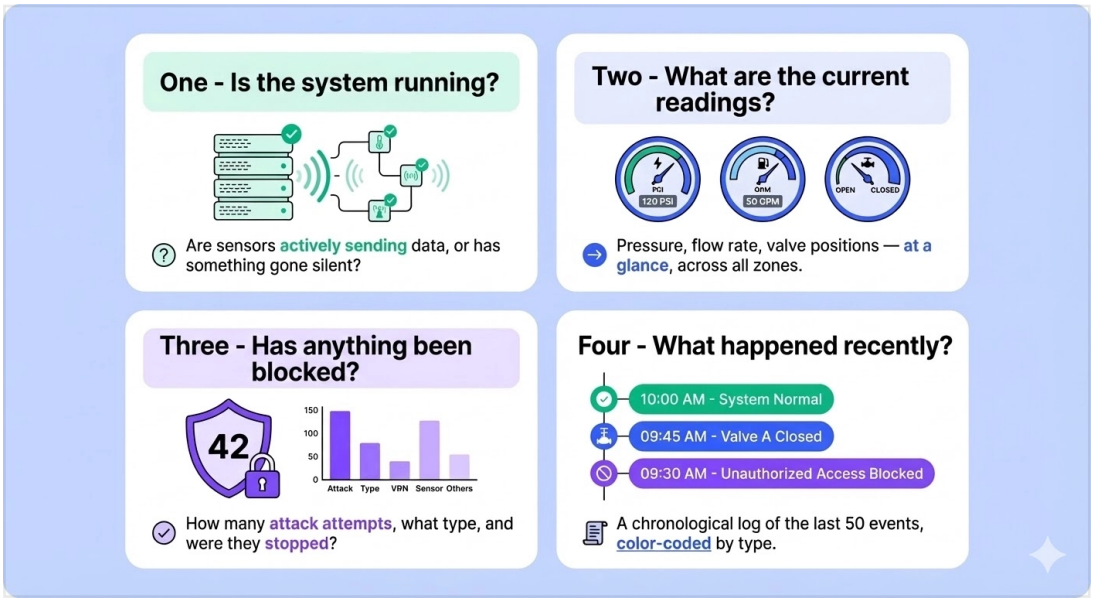
debugging. It works for development. It doesn't work when someone needs to understand a situation in five seconds.

Three Levels of Visibility

Each level builds on the one before it. The dashboard doesn't replace your subscriber's validation logic. It just gives it a face.



What the Operations Team Needs



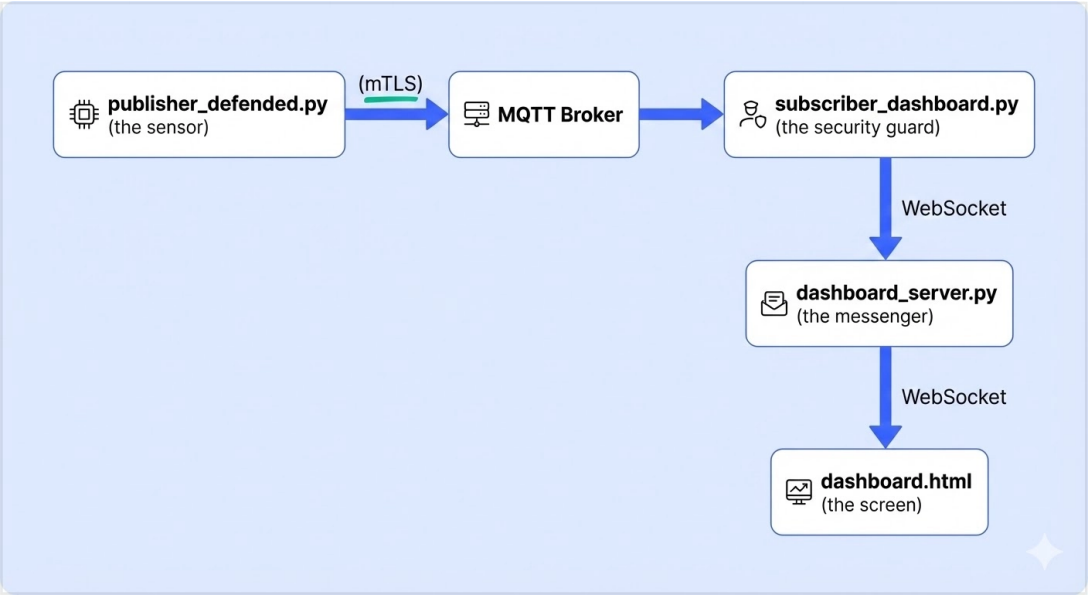
The technology differs — some use Grafana, some use Splunk, some use custom tools. The dashboard you're about to build will use WebSocket and HTML. A production SOC might use Elasticsearch and Kibana. But the principle is the same: turn raw data into something a human can act on quickly.

💡 What's a SOC?

A Security Operations Center (SOC) is a room full of analysts watching dashboards

24/7, monitoring an organization's networks for threats. Think of it as mission control for cybersecurity. Large companies like IBM, CrowdStrike, and Palo Alto Networks run SOCs that watch millions of events per day. Your Grand Marina dashboard is a miniature version of the same idea — one screen, one pipeline, the same principles.

How the Dashboard Works



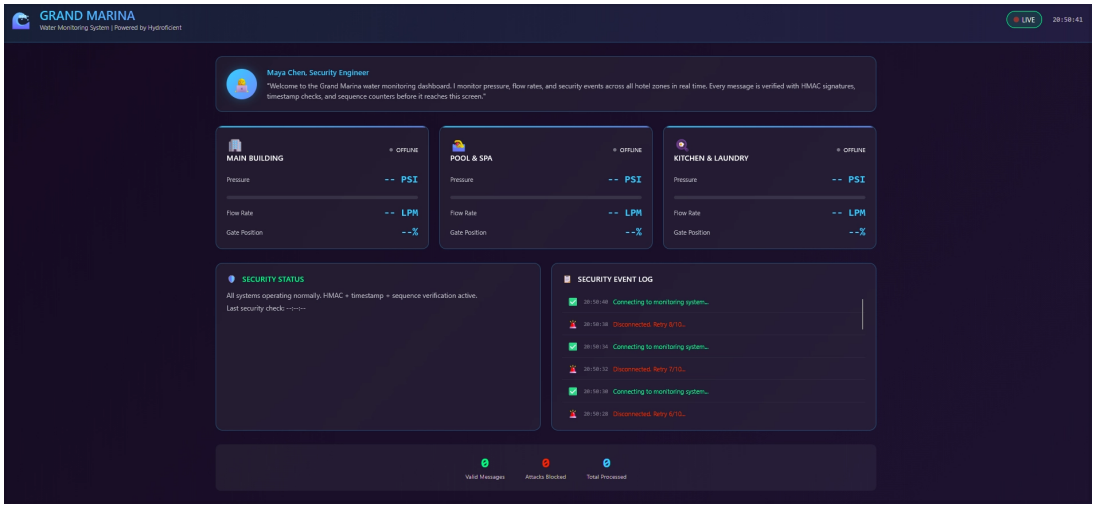
What's new is the last two steps: after the subscriber makes its accept/reject decision, it tells the dashboard server, which pushes the event to your browser in real time. You're not rewriting anything. You're adding visibility. I have added the files in the shared drive. Open them later once you are back at your seat.
attack_simulator.py

Component 1: dashboard.html — "The Screen"
This is what you see when you open your browser. It's a single HTML file with built-in CSS and JavaScript. No frameworks, no build tools, no npm install — just one file that runs in any modern browser.
The dashboard has five visual sections:

- **Header** — shows "GRAND MARINA," a live clock, and a green "LIVE" indicator that turns red if the WebSocket disconnects
- **Maya section** — a welcome message from Maya for the operator
- **Zone cards** — three cards for Main Building, Pool & Spa, and Kitchen & Laundry. Each one shows pressure, flow rate, and gate position for that area
- **Security section** — a status panel that shows "All systems operating normally" in green or "ATTACK DETECTED" in red, plus a scrolling event

log
- **Stats footer** — three numbers at the bottom: valid messages, attacks blocked, total processed

The JavaScript in that HTML file does one thing: it opens a WebSocket connection to ws://localhost:8765 and listens for messages. When a message arrives, it updates the right part of the screen. Valid messages update the zone cards with new sensor values. Attack messages trigger the red alert panel and shake the affected zone card.



Component 2: dashboard_server.py **— "The Messenger"**

This is the bridge between Python and the browser. It runs two servers:

- **HTTP server on port 8000** — serves dashboard.html when you visit http://localhost:8000 in your browser. Simple file server — just hands the HTML file to the browser.
- **WebSocket server on port 8765** — keeps an open connection with the browser. When the subscriber calls log_valid_message() or log_rejected_message(), this server broadcasts the event to every connected browser tab.

Why WebSocket instead of regular HTTP? Think of HTTP like a walkie-talkie: the browser asks a question, the server answers, and the conversation is over. To get the next update, the browser has to ask again. WebSocket is like a phone call — once the connection opens, both sides can talk whenever they want. The server can push new data to the browser the instant it arrives, without the browser asking.

This matters for a live dashboard. Sensor data arrives every 5 seconds. Attacks can happen at any moment. You don't want the browser polling the server asking "anything new?" every second. You want the server to push events the moment they happen.

Component 3: subscriber_dashboard.py**— "The Security Guard"**
This is the most important file. It combines everything from Week 6's (Project 6) subscriber_defended.py with the dashboard integration. The validation logic is identical:

- **HMAC check** — is the signature valid? Catches message tampering.
- **Timestamp check** — is the message fresh? Catches stale replays.
- **Sequence check** — is this a new message? Catches recent replays.

What's new is what happens *after* validation. In Week 6 (Project 6), the subscriber just printed to the terminal. Now it also calls the dashboard server.

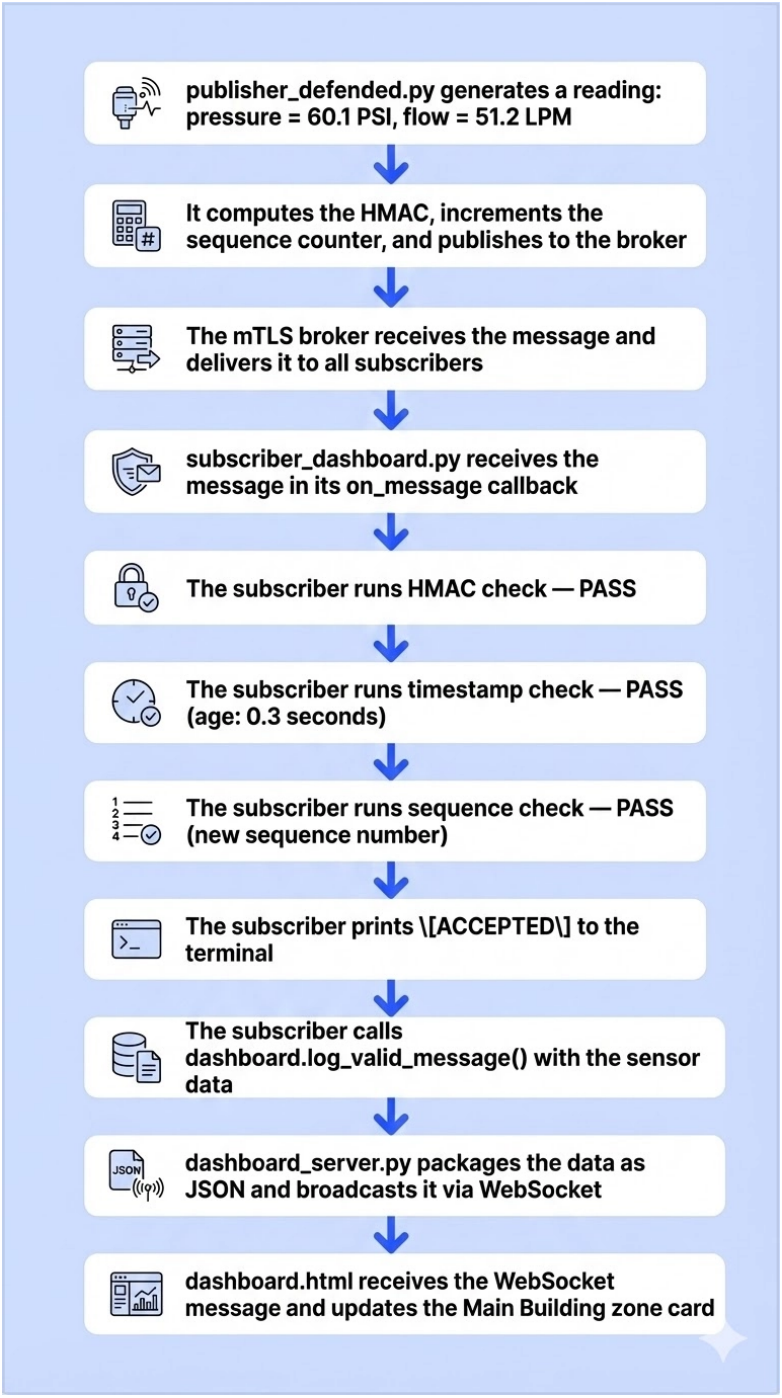
```
# When a message passes all checks:
dashboard.log_valid_message(device_id, sensor_data, topic)

# When a message fails any check:
dashboard.log_rejected_message(reason, attack_type, source,
topic)
```

The terminal output is still there — you'll see the same ACCEPTED/REJECTED lines as before. The dashboard is an addition, not a replacement.

Think of it this way: the subscriber is a security guard checking IDs at the door. In Week 6 (Project 6), the guard checked IDs and shouted the results into an empty hallway. Now there's also a security camera feed going to a monitor at the front desk. Same guard, same checks, different audience.

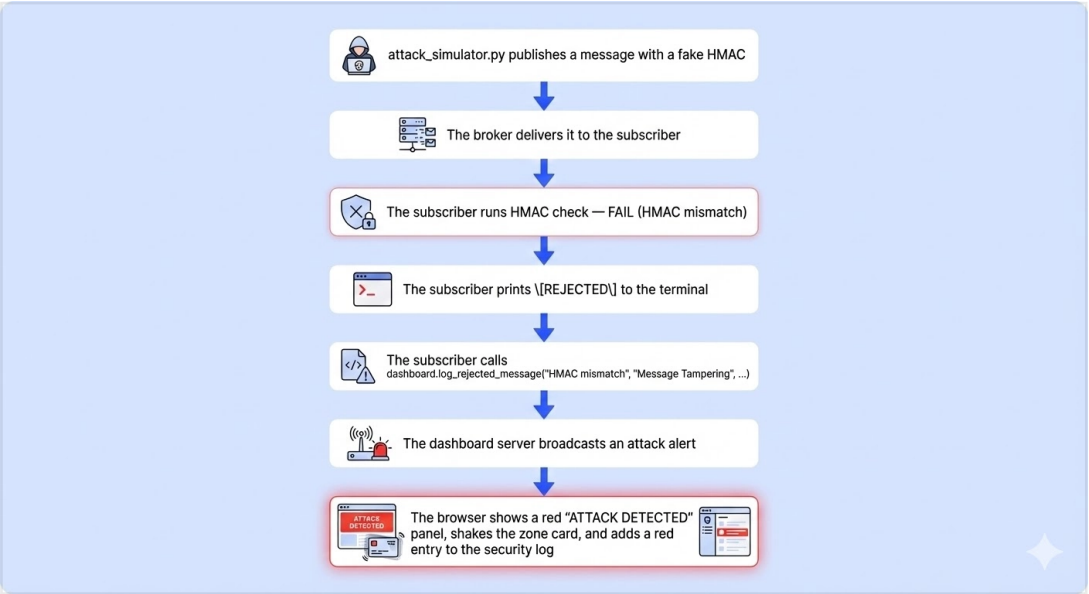
Component 4: publisher_defended.py — "The Sensor"
This one's easy. It's the same publisher as Project 6. Hasn't changed. It still connects to the mTLS broker with its device certificate, generates simulated sensor readings, signs each message with HMAC, increments the sequence counter, and publishes.
The publisher doesn't know the dashboard exists — and it doesn't need to. It just sends data. The subscriber handles everything else.

How a Message Flows Through the System



Total time from publish to browser update: less than a second.

Now trace an attack:



File	Don't Touch	Your Territory
dashboard.html	WebSocket connection logic, message handlers	Title, colors (CSS variables), zone names, Maya's message
dashboard_server.py	WebSocket server, HTTP server, broadcast logic	Nothing — this is infrastructure
subscriber_dashboard.py	HMAC/timestamp/ sequence validation, MQTT setup	Alert thresholds (e.g., high-pressure warnings)
publisher_defended.py	Everything — don't change it	Nothing — it stays as-is from Week 6 (Project 6)

🏭 **How This Maps to Production Systems**

In a production IoT deployment, these same four roles exist but with different names. The "publisher" is a real sensor or gateway device. The "broker" is a cloud service like AWS IoT Core or Azure IoT Hub. The "subscriber" is a backend service that processes and validates messages. And the "screen" is a dashboard built with Grafana, Kibana, or a custom web app. The architecture you're learning here — publisher, broker, processor, dashboard — is the standard pattern.

Launch the Dashboard

Follow the Steps Below

- **Part 1: Install the WebSocket Library (2 minutes)**
  - The dashboard needs one new Python package. In any terminal:

```
pip install websockets
```

- **Part 2: Start the Dashboard Subscriber (5 minutes)**
  - Make sure the mTLS broker you started in the Prerequisites is still running in Terminal 1 (mosquitto -c mosquitto_mtls.conf -v).
  - Then open **Terminal 2** and start the subscriber with dashboard integration:

```
python subscriber_dashboard.py
```

  - **What you should see:**

```
============================================================
Grand Marina Security Dashboard (Live)
============================================================
Subscribing to: hydroficient/grandmarina/#
Certificate:    certs/device-001.pem
Max message age: 30 seconds
Dashboard:       <http://localhost:8000>
============================================================

[CONNECTING] localhost:8883...
```

  - After a moment, a browser tab should open automatically to http://localhost:8000. If it doesn't, open your browser manually and go to that address.
    - **What you should see in the browser:**

```
The Grand Marina dashboard with a dark background
Three zone cards showing "OFFLINE" (no data yet)
A "LIVE" indicator in the top right (green dot pulsing)
Maya's message at the top
An empty security event log
Stats footer showing all zeros
```

  - The subscriber terminal should then show:

```
[SUCCESS] Connected to broker as GrandMarina-Dashboard-Live
[INFO] Replay defenses ACTIVE
[INFO]   HMAC verification: ON
[INFO]   Timestamp window: 30 seconds
[INFO]   Sequence tracking: ON
[INFO]   Live dashboard: ON
[INFO] Subscribing to: hydroficient/grandmarina/#
[SUBSCRIBED] QoS granted: (1,)

[LISTENING] Waiting for messages (Ctrl+C to stop)...
```

- **Part 3: Start the Publisher**
  - Open **Terminal 3** and start the defended publisher from Project 6:

```
python publisher_defended.py
```

    - **What you should see in Terminal 3:**

```
============================================================
HYDROLOGIC Sensor Publisher (Defended)
============================================================
Device ID: 001
Topic: hydroficient/grandmarina/device-001/sensors
Certificate: certs/device-001.pem
Defenses: timestamp + sequence counter + HMAC-SHA256
============================================================

[CONNECTING] localhost:8883...
[SUCCESS] Connected to broker as HYDROLOGIC-Device-001
[INFO] Replay defenses ACTIVE: timestamp + sequence + HMAC

[PUBLISHING] Sending defended readings (Ctrl+C to stop)...

[1] Published: 51.23 LPM | seq=1 | hmac=a3b7c9d1e5f2.××
[2] Published: 49.87 LPM | seq=2 | hmac=7f2e4a6b8c01.××
```

- **What you should see in Terminal 2 (subscriber):**

```
[ACCEPTED] Device: HYDROLOGIC-Device-001 | Flow: 51.23 LPM |
Seq: 1
  HMAC: PASS | Timestamp: PASS (Age: 0.3s (max: 30s)) |
Sequence: PASS
  Stats: 1 accepted, 0 rejected (1 total)
```

- **What you should see in the browser:**
  - This is the moment. Switch to your browser tab and watch:
    1. The **Main Building** zone card changes from "OFFLINE" to "ONLINE" (green dot pulses)
    2. **Pressure** updates to show a live PSI value
    3. **Flow Rate** updates to show LPM
    4. **Gate Position** updates with a percentage
    5. The **pressure bar** fills up proportionally
    6. The **security event log** shows green entries: "Main Building: pressure=60.1 PSI, flow=51.2 LPM"
    7. The **stats footer** increments: Valid Messages go up by 1 every 5 seconds
  - Leave it running for 30 seconds and watch the numbers update in real time.
- **If Nothing Appears in the Browser**
  - The most common reason is that the publisher is sending to a different topic than the subscriber expects. Check that your publisher's topic is hydroficient/grandmarina/device-001/sensors and that your subscriber is subscribing to hydroficient/grandmarina/#. Also make sure the browser tab is pointed at

http://localhost:8000 (not https).

- **Part 4: Test the Connection (5 minutes)**
  - Let's verify the dashboard responds to changes:
    - **Test 1: Stop the publisher.** Go to Terminal 3 and press Ctrl+C. Watch the dashboard — the zone card stays on its last reading, but no new events appear in the log. The "LIVE" indicator stays green because the subscriber is still connected.
    - **Test 2: Restart the publisher.** Run python publisher_defended.py again in Terminal 3. Watch the dashboard — new events start flowing into the log. The zone card values update. The valid message counter keeps going up.
    - **Test 3: Check the stats.** Look at the stats footer. It should show something like:

Stat	Value
Valid Messages	8+ (depending on how long you ran it)
Attacks Blocked	0
Total Processed	Same as Valid Messages

- The "Attacks Blocked" counter is zero because you haven't run any attacks yet. That comes in Step 2.

- **Part 5: Submission**
  - Take a screenshot of the dashboard with live data flowing.
  - You'll also use this in your capstone presentation.

Step 2: Live Attack Demo

Anatomy of the Attack Simulator

This runs a three-phase attack against your defended pipeline. It connects to the broker using valid mTLS credentials — think insider threat. Someone who has a legitimate certificate but is using it maliciously. It tries to break through your defenses. The subscriber validates every message. The dashboard shows the results in real time. The expected outcome: every attack is blocked. The dashboard lights up with red alerts, zone cards shake, and the stats bar counts each rejected attempt. That's seven weeks of work, defending your pipeline live.

The Red Team and Blue Team mean in our industry?
In the cybersecurity industry, the team that attacks is called the "red team" and the team that defends is called the "blue team." Penetration testers (pen testers) are hired by companies to be the red team — they try to break in so the blue team can fix the gaps before a real attacker finds them. Your attack simulator is a

miniature red team exercise. You built the defenses (blue team, Projects 3-6) and now you're testing them with a simulated attacker (red team, the simulator). Professional pen testers at firms like CrowdStrike and Rapid7 do the same thing at much larger scale.

Phase 1: Eavesdrop
The simulator subscribes to hydroficient/grandmarina/# and intercepts legitimate messages from your publisher. It displays them in the terminal with a dramatic "INTERCEPTED" box showing the topic, pressure, flow rate, and timestamp.

What this demonstrates: An attacker with valid mTLS credentials can read all messages on the broker. TLS protects messages on the wire (an outsider can't sniff the network), but someone with a valid client certificate can subscribe to any topic. This is the insider threat scenario from Project 6.	**What the dashboard shows during this phase:** Nothing unusual. The eavesdropper is just listening — it's not injecting anything. Your publisher's messages still flow normally. The dashboard shows green checkmarks in the event log as usual.	**Why it matters:** Eavesdropping is the first step in most real attacks. The attacker learns the message format, the topic structure, and the data patterns before attempting anything active. In the cybersecurity industry, this is called "reconnaissance" — the attacker gathers intelligence before doing anything destructive. The MITRE ATT&CK framework, which is the standard reference for cataloging attack techniques, lists reconnaissance as the very first phase of any cyberattack.

Phase 2: Inject Fake Data
The simulator publishes a message with:

- A valid-looking device ID (HYDROLOGIC-Device-001)
- A current timestamp
- A high sequence number (99999)
- Dangerous sensor values (250 PSI pressure — double the normal range)
- A fake HMAC signature (FAKE_HMAC_0000...)

What the subscriber does: The first validation check is HMAC verification. The subscriber computes what the HMAC *should* be using the shared secret and compares it to the fake one in the message. They don't match. The message is rejected immediately — the timestamp and sequence checks never even run.	**What the dashboard shows:** A red "ATTACK DETECTED" alert appears. The attack type shows "Message Tampering." The affected zone card shakes. A red entry appears in the security log: "BLOCKED: Message Tampering from HYDROLOGIC-Device-001."	**What this proves:** Even though the attacker has valid mTLS credentials to connect to the broker, they can't forge sensor data because they don't know the HMAC shared secret. The HMAC defense you built in Project 6 catches this.

Phase 3: Replay

The simulator re-sends one of the messages it captured during Phase 1. This is a real captured message — it has a valid HMAC, because it was originally signed by the real publisher. But by the time it's replayed, two things have gone wrong:

1. **The timestamp is stale.** Several seconds have passed since the original message was sent. The subscriber's 30-second freshness window may catch this, especially if the eavesdropping phase ran long enough.
2. **The sequence number is old.** The subscriber already accepted the original message and recorded its sequence number. Seeing the same number again triggers the replay defense.

What the subscriber does: It checks HMAC first, and it passes, because the message is genuine. Then it checks the timestamp — if enough time passed, this fails. If the timestamp still falls within the window, it checks the sequence counter — and that definitely fails, because the subscriber already recorded a higher sequence number.	**What the dashboard shows:** Another red alert, this time showing "Replay Attack" or "Stale Message" depending on which check triggers first. Another red entry in the security log. Another increment to the "Attacks Blocked" counter.	**What this proves:** Even a perfectly valid captured message can't be replayed. The timestamp and sequence defenses from Project 6 catch it. This is the scenario from Project 6 — "the same sensor reading, published twice" — and now you can watch it get blocked in real time.

That's all three phases. So how do you know if your defenses actually held up? Check the numbers.

The Scoreboard

After the simulator finishes, look at the dashboard's stats footer:

Stat	What It Shows
Valid Messages	All the legitimate messages from your publisher that passed all three checks
Attacks Blocked	The injection attempt + the replay attempt (at least 2)
Total Processed	Valid + blocked combined

Your subscriber's terminal will show the same counts. The dashboard just makes them visible at a glance. Numbers tell you *what* happened. But the real question is *why* every attack failed — and which defense caught each one.

Why Every Attack Fails

Here's the mapping from attack to defense:

Attack Phase	What the Attacker Tries	Which Defense Catches It	Why It Works

Phase 1: Eavesdrop	Listen to all messages	None (passive)	Listening isn't blocked — but the attacker can't act on what they hear
Phase 2: Inject	Publish with fake HMAC	HMAC verification	Attacker doesn't know the shared secret
Phase 3: Replay	Re-send a captured message	Sequence counter (and/or timestamp)	Subscriber tracks the highest sequence seen per device

Notice that Phase 1 has no defense listed. That's intentional — listening isn't an active attack. In production, you'd limit which clients can subscribe to which topics using broker ACLs (access control lists). For this exercise, the attacker can listen, but they can't do anything useful with what they hear.
No single defense would stop everything. HMAC alone wouldn't catch a replayed message (the HMAC is valid). Timestamps alone wouldn't catch a replay within the 30-second window. Sequence counters alone wouldn't catch a message from a different device. The three layers work together, and the dashboard lets you see them working together in real time.

Run the Live Attack Demo

Follow the Steps Below

- Part 1: Run the Attack (10 minutes)
  - Open **Terminal 4** and run the attack simulator:
    - python attack_simulator.py
- **Part 2: Watch the Dashboard (10 minutes)**
  - While the attack runs, switch to your browser. Here's what you should see at each phase:
  - **During Phase 1 (Eavesdrop):** Nothing changes. The eavesdropper is only listening. Your publisher's normal data still flows through with green checkmarks.
  - **During Phase 2 (Inject):**
    - The **security status panel** turns red: "ATTACK DETECTED"
    - The **attack type** shows: "Message Tampering"
    - The **attack source** shows: "HYDROLOGIC-Device-001"
    - The **Main Building zone card shakes** with a red border
    - The green "BLOCKED BY SECURITY SYSTEM" badge appears
    - A **red entry** appears in the security event log

- **During Phase 3 (Replay):**
  - Another **red alert** appears (it may say "Replay Attack" or "Stale Message")
  - The zone card **shakes again**
  - Another **red entry** in the event log
- After the attack finishes, the security panel returns to green: "All systems operating normally."

- **Part 3: Check the Results (5 minutes)**
  - Look at the **stats footer** at the bottom of the dashboard:

Stat	Expected
Valid Messages	10+ (depends on how long the publisher ran)
Attacks Blocked	2+ (injection + replay)
Total Processed	Valid + Blocked combined

Now check **Terminal 2** (the subscriber). You should see the rejection reasons:

- Fill in this results table from your own run:

Attack Phase	What Was Attempted	Blocked?	Which Defense Caught It
Phase 2: Injection	Fake sensor data with bogus HMAC	Yes	HMAC verification
Phase 3: Replay	Re-sent captured message	Yes	Sequence counter

If both attacks were blocked, your pipeline is fully defended.

- **Part 4: Take Your Screenshots (5 minutes)**
  - You'll need these for your final presentation. Capture:
    - **Dashboard with attack alert visible** — the red "ATTACK DETECTED" panel with the "BLOCKED BY SECURITY SYSTEM" badge. You may need to run the attack simulator again to capture this at the right moment.
    - **Dashboard stats footer** — showing both valid messages and attacks blocked.
    - **Terminal output** — the subscriber showing REJECTED messages with reasons.
  - **Screen recording shortcuts:**
    - **Windows:** Press Win+G to open Game Bar, then click the record button
    - **macOS:** Press Cmd+Shift+5 to open the screenshot toolbar, then choose "Record Selected Portion."

    - (Optional) If your computer supports screen recording or GIF capture, record a short clip of the attack demo running while the dashboard is visible. A recording of the live attack demo makes your presentation more convincing than a static screenshot.
    - **Tip:** Run the attack simulator a second time to practice capturing the screenshot at the right moment. The red alert stays visible for about 5 seconds.

Customize Your Dashboard

Follow the Steps Below

- Part 1: Change the Dashboard Title
    - Open dashboard.html in your text editor. Find the header section near the top of the HTML body:

```
<h1>GRAND MARINA</h1>
<p>Water Monitoring System | Powered by Hydroficient</p>
```

        - Change the title and subtitle to something of your own. A few ideas:

```
Your name: <h1>ALEX'S SECURITY HQ</h1>
A creative name: <h1>AQUA SHIELD</h1>
The hotel name with your team: <h1>GRAND MARINA - TEAM ALPHA</
h1>
```

    - Also, find Maya's name if you want to customize her character:

```
<div class="maya-name">Maya Chen, Security Engineer</div>
You could change this to your own name and title:
<div class="maya-name">Your Name, Junior Security Engineer</
div>
```

    - **Save the file.** Refresh your browser (Ctrl+R or Cmd+R). The title should update immediately — no need to restart the Python scripts.
- **Part 2: Change the Color Scheme**
    - At the top of the <style> section in dashboard.html, find the CSS variables block marked with a comment:

```
/* CUSTOMIZABLE COLORS */
:root {
    --bg-primary: #0a0a1a;
    --bg-secondary: #12122a;
    --accent-cyan: #00d4ff;
    --accent-blue: #4a9eff;
    --accent-green: #00ff88;
    --accent-red: #ff4444;
    --accent-orange: #ff9500;
    ...
}
```

    - Try changing one or two accent colors. Here are some tested combinations:
        - **Purple theme:**

```
--accent-cyan: #b388ff;
--accent-blue: #7c4dff;
--border-glow: rgba(179, 136, 255, 0.3);
```

        - **Amber/gold theme:**

```
--accent-cyan: #ffc107;
--accent-blue: #ff9800;
--border-glow: rgba(255, 193, 7, 0.3);
```

        - **Teal theme:**

```
--accent-cyan: #00bfa5;
--accent-blue: #009688;
--border-glow: rgba(0, 191, 165, 0.3);
```

        - Pick a combination or experiment with your own hex colors. Save and refresh the browser after each change to see the result. Here are some free tools that you can use to find **Hex Colors:**
    - **Tip:** Only change the accent colors at first. Changing the background colors (--bg-primary, --bg-secondary) can make text hard to read if you pick something too light.
- **Part 3: Rename or Modify a Zone**
    - Find the zone cards in the HTML. Each zone card has a title like:

```
<div class="sensor-title">MAIN BUILDING</div>
and an icon:
<div class="sensor-icon">&#127970;</div>
```

    - You can rename zones to match your own scenario. For example, if your hotel has a rooftop bar:

```
<div class="sensor-icon">&#127865;</div>
<div class="sensor-title">ROOFTOP BAR</div>
```

    - Common HTML emoji codes you can use:

```
&#127970; = office building
&#127946; = swimmer
&#127859; = cooking
&#127865; = cocktail glass
&#127793; = plant/garden
&#128692; = gym/bicycle
```

    - **Note:** If you rename zones, the data from the publisher will still flow into the "Main Building" card because the topic routing maps to main_building by default. The zone card will work correctly — it just has a different label.
- **Part 4: Add a Custom Alert Rule**
    - This part goes beyond HTML into the subscriber's Python code. Open subscriber_dashboard.py and find the on_message function. Look for

the section that handles accepted messages:

```
if accepted:
    stats["accepted"] += 1
    # ... terminal print ...
    if dashboard:
        sensor_data = data×get("readings", {})
        dashboard.log_valid_message(device, sensor_data,
msg.topic)
```

- Right after the dashboard call, add a custom high-pressure alert:

```
if accepted:
    stats["accepted"] += 1
    print(f"\\n[ACCEPTED] Device: {device} | Flow: {flow} LPM |
Seq: {seq}")
    print(f"  HMAC: PASS | Timestamp: PASS
({results['timestamp']['detail']}) | Sequence: PASS")

    if dashboard:
        sensor_data = data×get("readings", {})
        dashboard.log_valid_message(device, sensor_data,
msg.topic)

    # --- YOUR CUSTOM ALERT ---
    pressure = data×get("readings",
{}).get("pressure_upstream", 0)
    if pressure > 65:
        print(f"  [ALERT] High pressure detected: {pressure}
PSI")
```

- This adds a terminal alert when pressure exceeds 65 PSI. Since the publisher generates random pressure between 58-62 PSI, you'll occasionally see this alert if you lower the threshold to 60:

```
if pressure > 60:
        print(f"  [ALERT] High pressure detected: {pressure}
PSI")
```

- **Save the file.** Stop the subscriber (Ctrl+C in Terminal 2) and restart it:
  python subscriber_dashboard.py
  Watch the terminal — you should see your custom alert appear on some messages.

- **Part 5: Deliverable**
  - Take a screenshot of your customized dashboard showing:
    - **Your custom title/branding** visible in the header
    - **Your color scheme**

💡 **Going Further (Optional)**

If you want a bigger challenge, try these:

- Add a second alert rule for low flow rate (e.g., flow below 45 LPM could indicate a pipe blockage)
- Send the custom alert to the dashboard by calling dashboard.log_rejected_message() with a new attack type like "Anomaly"
- Add a counter that tracks how many high-pressure readings you've seen in a row

These are stretch goals — they're not required for the deliverables, but they show deeper understanding of the code.

Step 3: Create the Final Presentation

Building Your Final Presentation

The Template

Open the capstone presentation template in Google Slides and go through it:
Make a copy of this template (**File > Make a copy**) so you have your own version to edit.
The template has 11 slides, plus one optional bonus slide for a security analysis chart. Some are partially filled in to guide you. Others are blank — that's where your work goes.

You must be wondering what the need for a template is. Everyone can create their own template, right? Here is the answer:

In consulting and security work, you almost never start a presentation from a blank slide. Teams use templates to ensure consistency, save time, and focus their energy on the content rather than the design. The template gives you the structure. Your job is to fill it with evidence and insights that only you have.

Slide-by-Slide Walkthrough

Slide 1: Title Slide

Your name, the project title, and the Hydroficient branding. Fill in your name and any subtitle you want. Simple — make it yours.

Slide 2: The Threat

Three cards — Eavesdrop, Inject Fake Data, Replay Attack. The card titles are already there. You need to describe what each attack does and why it's dangerous, in your own words. There's also a summary line at the bottom asking

why unprotected IoT pipelines are vulnerable.

Why this slide: Always start with the problem. Before anyone can appreciate what you built, they need to understand what you were up against. Don't just list the attack names — explain what each one actually does and why it's dangerous.

Slide 3: Defense Architecture
Four defense layer boxes labeled Defense 1 through Defense 4. For each one, fill in the name of the defense (e.g., TLS, mTLS, HMAC, Timestamps & Sequence Counters) and explain what it does and which attack it stops. At the bottom, explain why you need all four — what happens if you remove one?

Why this slide: Now that you've laid out the threats, the natural next question is — what did you do about them? This is where you show the solution you built. The key point to make: every layer matters. If you remove one, a specific attack gets through. That's what makes this architecture strong.

Slide 4: Experiment Setup
Two sections side by side. On the left, insert a diagram of your experiment setup (how the publisher, broker, subscriber, attacker, and dashboard connect). On the right, describe your setup process step by step — what components were running, how you launched the attack simulator, and what you were monitoring.

Why this slide: You've shown the threats and the defenses — now explain how you actually put them to the test. Without this context, the results on the next slide have nothing to stand on. Walk the audience through your setup so they can trust the findings.

Slide 5: Attack Results
A table with four columns: Attack Phase, What the Attacker Tried, Defense That Caught It, and Why It Works. The phase names (Eavesdrop, Inject, Replay) are filled in. You fill in the rest from your experiment results.

Why this slide: This is where your experiment data speaks for itself. You ran the attacks, the defenses caught them — now present that evidence clearly. Which attacks were attempted, what caught them, and why it worked.

Bonus Slide: Analysis Chart (Optional)
If you want to make your presentation stand out, consider adding a chart slide right here — between your attack results and your screenshots. You already have the data from your experiments across Projects 4-6. A bar chart showing defense effectiveness (rejection rates across your 12 test configurations from Project 6) is probably your most powerful visual. You could also chart TLS latency impact (with vs. without encryption from Project 4) or mTLS overhead comparison (one-way vs.

mutual TLS from Project 5). Pick whichever dataset tells the clearest story — one strong chart beats three cluttered ones.

Why this slide: Numbers in a table are good. Numbers in a chart are better. A well-chosen chart turns your raw experiment data into a visual argument that lands instantly. This slide isn't required — your template works fine without it — but it's the kind of extra effort that signals you can communicate data, not just collect it.

Slides 6-8: Screenshots
Three slides for visual evidence from your dashboard:

- **Slide 6** — Dashboard during an active attack (red alerts, shaking zone cards, rejection logs)
- **Slide 7** — Stats footer showing attacks blocked vs. valid messages accepted
- **Slide 8** — Your customized dashboard (custom title, color theme, renamed zones)

For each slide, insert a screenshot or screen recording from your own dashboard. If you have a video of your dashboard in action (attacks being blocked live, zone cards updating in real time), use that — video is even more impactful than a static screenshot. Wherever you can add video, do it.

Why these slides: Talking about results is one thing — showing them is another. These slides are your proof. Screenshots and videos of your dashboard blocking attacks in real time, your stats, and your custom design show that you actually built this and that it works.

Slide 9: Key Insights
Four numbered insights. The first one is filled in as an example: *"No single defense is enough."* The other three are yours to write. Think about what surprised you, what you learned, and what you'd do differently.

Why this slide: You've shown what happened — now share what you took away from it. This is where you go beyond the data and show that you can think critically about security, not just follow instructions.

Slide 10: Recommendations
Four numbered recommendations for Grand Marina. All are placeholder prompts. Think about what the hotel should do next — deploy to production? Add monitoring? Create incident response plans? Automate certificate rotation? These should be specific and actionable.

Why this slide: This is the slide that separates someone who can execute tasks from someone who can advise on what should happen next. Think beyond the current project — what would you recommend if this were a real engagement?

(Optional) Want to Take Your Presentation Further?
If you want to dig deeper into how consultants structure and deliver presentations, here are a few techniques worth knowing:

- **The Pyramid Principle** — Lead with your conclusion, then show the evidence. Don't make the audience wait until the end to find out what you're going to say. Read more here.
- **Situation-Complication-Resolution (SCR)** — Structure your story as: here's what exists → here's what's wrong → here's what we did about it. Your slides already follow this arc.
- **Headlines should be findings, not labels** — "Experiment Results" is a label. "All attacks blocked with combined defenses" is a finding. If someone reads only your slide titles, they should understand the full story.
- **Translate for the audience** — "HMAC-SHA256 verification" means nothing to a CTO. "Every message is verified as authentic and untampered" does. Always translate technical terms into language the audience can act on.

Using AI to Build Your Slide Content

The Rule: You Bring the Content, AI Brings the Polish
This is the most important thing to understand before you start prompting.
You are not asking AI to create your presentation. You are asking it to help you say what you already know more clearly. The difference matters:

What You Do	What AI Does
Decide what each slide says	Polish the language
Provide your actual experiment data	Translate technical terms for the audience
Write rough bullet points from memory	Turn rough bullets into clean, confident sentences
Review every line for accuracy	Suggest better phrasing or structure
Cut anything that doesn't sound like you	Follow your tone and style instructions

Part 1: Write Rough Notes First
Before you open ChatGPT or Claude, spend 10 minutes writing rough notes for each slide. These don't need to be polished — they just need to be real. Here's what that looks like:

Slide 2 (The Threat) — your rough notes might be:	**Slide 9 (Key Insights) — your rough notes might be:**
Eavesdrop - anyone on same network can read all sensor data, plaintext MQTT, saw this in project 3 when we used mosquitto_sub with # wildcard	No single defense is enough - timestamp alone misses immediate replays, counter alone misses modified messages

That's all the AI needs. Real observations from your actual work.

Part 2: Use ChatGPT or Claude for Slide Language

Once you have rough notes for your slides, open **ChatGPT** or **Claude**. Either works well for this. Don't start with "write my presentation." Build your prompt in layers — the same approach you learned in Project 4.

Step 1: Set the context

Start by telling the AI who you are, what you're presenting, and who the audience is:

"I'm a cybersecurity extern presenting my capstone project to the head of a technology company. My presentation covers a security pipeline I built for a hotel's IoT water monitoring system. The audience is technical enough to understand concepts but doesn't want jargon. The tone should be confident and professional — like a junior consultant presenting findings."

Step 2: Share the template presentation

Tell the LLM exactly which task you want it to perform.

"Help me write clean, concise language for my presentation slides. Each slide should deliver one clear message. Use headlines that state a finding, not a label — for example, 'All Attacks Blocked with Combined Defenses' instead of 'Experiment Results.' I'll give you each slide number with my rough notes, and I want you to return polished language I can put directly on the slide."

Upload your presentation template so the AI can see the full slide structure. To upload the presentation, download your copied template as a PDF and upload it in the message window of the LLM. This gives it the complete picture of what you're working with — instead of you having to describe each slide from scratch.

Then list the slides you want help with. You don't need to do all 11 at once — start with the ones you find hardest to write.

Step 3: Feed your rough notes one by one

This is the most important step. Don't dump all 11 slides at once — work

through 2-3 slides per prompt. This keeps the AI focused and gives you a chance to course-correct between batches.
Start with the slides you find hardest to write:
"Let's start with Slides 2 and 3 (The Threat and Defense Architecture). Here are my rough notes for both. Turn my rough notes into clean slide language: a strong headline and 2-4 bullet points or short sentences per slide. Ask questions if you need any context or have any doubts."

Step 4: Review and revise
After each batch, read the output critically:

- **Is every claim actually from your notes?** If the AI added a statistic or incident you didn't mention, delete it. The AI doesn't know your experiment results — only you do.
- **Does it sound like you?** If a sentence feels like something a corporate brochure would say, rewrite it in your own voice. "Leveraging multi-layered cryptographic defenses" is not something you'd say. "Three defenses working together block every attack" is.

Ask the AI to revise anything that doesn't work:
"Slide 3 is too wordy. Cut each defense description to one sentence. Keep the headline."
"The language on Slide 9 sounds too generic. Make it more specific to my actual results — I tested 12 experiment configurations and only the 'all defenses' config achieved 100% rejection."